

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

12 de agosto de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Objetivos:

- 1. Velocidad, eficiencia, escalabilidad. Ley de Ahmdal**
- 2. Speedup, performance**
3. Modelos computacionales en paralelo - PRAM (1)
4. Modelos computacionales en paralelo - PRAM (2)
5. Particionamiento de datos y tareas
6. Random en paralelo
7. N-body, Divide y Vencerás (mergesort)

Al finalizar la sesión, los alumnos conocen:

1. Implicaciones de la ley de Amdahl/Gustafson para un algoritmo paralelo
2. Definir velocidad (speedup) y eficiencia de un algoritmo paralelo.
Noción de escalabilidad

**U2.1 Velocidad, eficiencia,
escalabilidad.**

Ley de Amdahl/Gustafson

Dado el **speedup** (velocidad):

$$S = \frac{T_s}{T_p}$$

En general, $S \leq p$ La **eficiencia** es la **carga promedio** por procesador:

$$E(n) = \frac{S}{p} = \frac{T_s}{p \cdot T_p}$$

En general, si $S \leq p$, $E \leq 1$.

Si $S=p$, $E=1$

Ley de Amdahl - strong scaling

Responde la pregunta sobre que tan rápido ejecuta un código en paralelo cuando utilizo p procesos. También llamado **strong scaling**, donde el tamaño del problema es constante (condición rígida, fuerte)

El **tiempo de ejecución secuencial** (normalizado a la unidad) es:

$T_s = f_s + f_p$, con la fracción de código secuencial f_s , y paralela f_p
($f_s + f_p = 1$)

Resolverlo en p procesadores, implica para el **tiempo de ejecución en paralelo**:

$$T_p = f_s + \frac{f_p}{p}$$

$$S = \frac{f_s + f_p}{f_s + f_p/p} = \frac{1}{f_s + f_p/p}$$

$$E = \frac{1}{p(f_s + \frac{1-f_s}{p})}$$

Para $p \rightarrow \infty$, $S = 1/f_s$ (limitado por la parte secuencial)

Ley de Amdahl - strong scaling

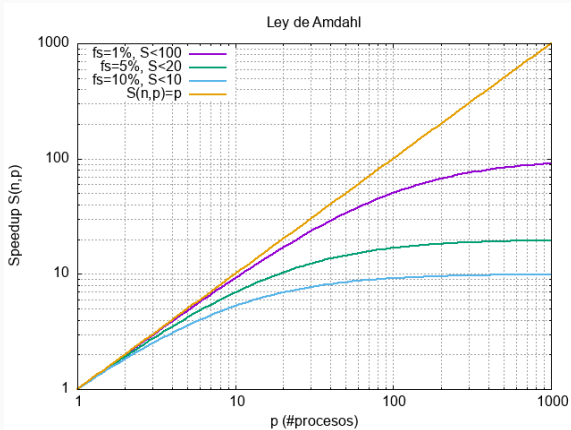


Figura 1: Ley de Amdahl

Dada una solución paralela, responde la pregunta sobre que más rápido se ejecuta en comparación a la versión secuencial, cuando el problema es de tamaño variable (creciente), en p procesos.

$$T_p = f_s + f_p$$

$$T_s = f_s + pf_p$$

$$S = \frac{f_s + pf_p}{f_s + f_p} = f_s + pf_p$$

$$E = \frac{f_s + pf_p}{p}$$

Speedup escala con el número de procesos p .

Ley de Gustafson - weak scaling

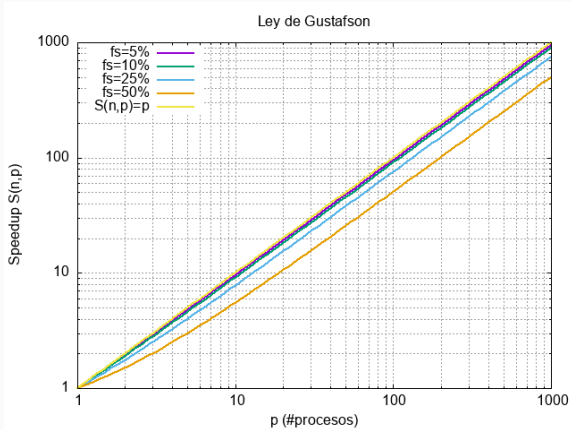


Figura 2: Ley de Gustafson

Ley de Amdahl/Gustafson

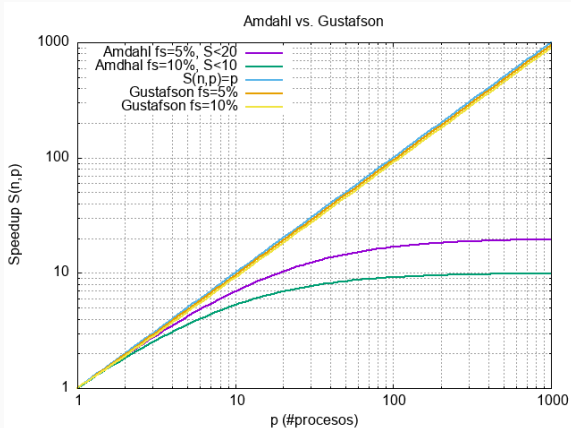


Figura 3: Ley de Amdahl vs. Gustafson

La **escalabilidad** evalúa el **performance** proporcional a un creciente número de procesos.

La **escalabilidad** mantiene una **eficiencia constante** si se **incrementa** el número de procesos (p), a la vez que la carga (n), es decir, si problemas grandes pueden ser resueltos con la misma eficiencia que problemas pequeños.

Esta propiedad no puede ser descrita por la Ley de Ahmdal, ya que esta última es independiente del número de procesos.

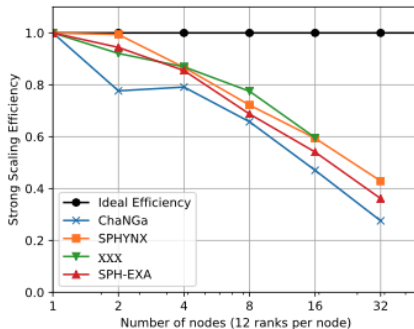


Figure 2: Parallel efficiency of ChaNGa(12 OpenMP tasks and 1 MPI rank per node), SPHYNX (12 OpenMP tasks and 1 MPI rank per node), XXX (12 MPI ranks per node) and SPH-EXA mini-app (12 MPI ranks per node) under strong scaling, in function of the number of nodes used (12 cores per node) on Piz Daint.

Figura 4: *Speedup de modelos SPH en 32 nodos*

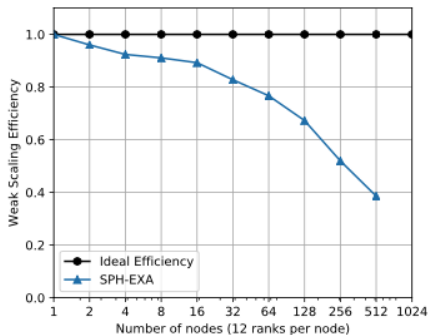


Figure 3: Parallel efficiency of the SPH-EXA mini-app (12 MPI ranks per node) under weak scaling, in function of the number of nodes used (12 cores per node) on Piz Daint.

Figura 5: *Speedup de modelos SPH en 1024 nodos*

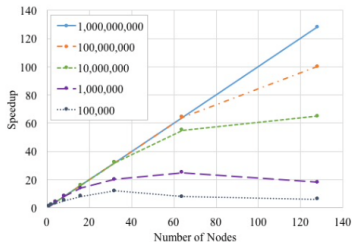


Figure 11: Strong scaling of the distributed N-Body application implemented with HPX on up to 128 nodes (20 cores per node). It shows a close to perfect HPX scalability for problem size of 10^9 particles.

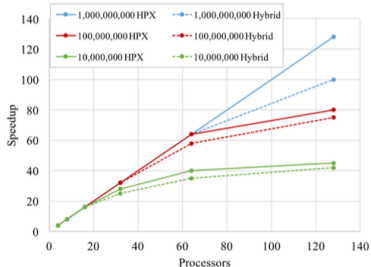


Figure 13: Comparison results of the strong scaling between HPX and a hybrid model (OpenMP/MPI), with up to 128 nodes (20 cores per node). The results illustrate a better performance for the HPX application for larger number the nodes, which is due to using a *future*-based request buffer between the remote nodes.

Figura 6: *Speedup de modelos NBody en 120 nodos*

Ejercicio: Eficiencia

Cuatro desarrolladores, tienen la tarea de paralelizar un algoritmo secuencial, disponiendo de un cluster con 1000 procesos para ello. Todos informan de ciertos problemas que afectan la escalabilidad:

- 4.1) El primero informa que utiliza el 80 % del tiempo en comunicación
- 4.2) El segundo informa que utiliza el 20 % del tiempo en comunicación
- 4.3) El tercero informa que no logra paralelizar el 10 % del algoritmo secuencial
- 4.4) El cuarto informa que no logra paralelizar el 90 % del algoritmo secuencial

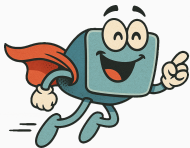
Estime la eficiencia en paralelo en cada caso ¿Cuál desarrollador logra una mayor eficiencia?

U2.2 Speedup, FLOPs

Speedup (FLOPs)

IEEE754 especifica un **Número de Coma Flotante** (FLOP) de precisión simple, de 32-bit base-2, con un valor máximo de $2^{128} \approx 3 \cdot 10^{38}$

Speedup es la **velocidad (máxima)** de ejecución del total de operaciones de coma flotante por unidad de tiempo, medida en **FLOPs** (Floating Point Operations per second). Se aplica a Aritmética de Coma Flotante simple (suma, multiplicación) Aritmetica compleja (división, raiz, trigonometría, logaritmos, etc) es mucho mas lenta y deben evitarse en HPC.



Microprocesadores

que generan dos o cuatro Double Precisión (FPs) por clock cycle, siendo la frecuencia entre 2-3 GHz, obtienen una velocidad de 4-12 **GFLOPs** por core.

Velocidad

de acceso a memoria se mide en **GByte/sec**, mientras que el cómputo se mide en **GFLOP/sec**.

Eliminación de Gauss - ¿Cuántos FLOP?

```
int main() {
    int n,i,j,k;
    cout.precision(4);
    cout.setf(ios::fixed);
    cout<<"\nIngrese cantidad de ecuaciones\n";
    cin>>n;
    float x[n];
    float** a = new float*[n];
    for (i=0;i<n;i++) a[i]= new float[n];

    // float a[n][n+1],x[n];
    cout<<"\nElementos de la matriz por fila:\n";
    for (i=0;i<n;i++)
        for (j=0;j<=n;j++)
            cin>>a[i][j];

    for (i=0;i<n;i++) //Pivot
        for (k=i+1;k<n;k++)
            if (abs(a[i][i])<abs(a[k][i]))
                for (j=0;j<=n;j++)
                {
                    double temp=a[i][j];
                    a[i][j]=a[k][j];
                    a[k][j]=temp;
                }

    cout<<"\nMatriz luego del pivot:\n";
    imprimir_matriz(a,n);
    for (i=0;i<n-1;i++) //gauss elimination
        for (k=i+1;k<n;k++){
            double t=a[k][i]/a[i][i];
            for (j=0;j<=n;j++)
                a[k][j]=a[k][j]-t*a[i][j]; //elementos bajo el pivot son cero o eliminados
        }

    cout<<"\n\nMatriz final:\n";
    imprimir_matriz(a,n);
    for (i=n-1;i>=0;i--){ //back-substitution ( x contiene x,y,z ..)
        x[i]=a[i][n]; //make the variable to be calculated equal to the rhs of the last equation
        for (j=i+1;j<n;j++)
            if (j!=i) //se substraen valores lhs excepto el coeficiente de la variable a calcular
                x[i]=x[i]-a[i][j]*x[j];
        x[i]=x[i]/a[i][i]; //se divide rhs por el coeficiente
    }
    cout<<"\nValor de las variables:\n";
    for (i=0;i<n;i++)
        cout<<x[i]<<endl; // se imprime x,y,z,...

    return 0;
}
```



Eliminación de Gauss, caso secuencial

¿Cuánto tiempo demorará resolver un sistema lineal de n ecuaciones?

Un sistema $Ax=b$ con n ecuaciones lineales necesita aproximadamente n^3 FLOP (Floating Point Operations) i.e., en un procesador de 1 GFLOPs, se necesitaría (**Tiempo de Cómputo**):



n	FLOP	tiempo (sec)
10	10^3	0.000001
100	10^6	0.001
1000	10^9	1.0
10000	10^{12}	1000 ($\approx$ 17 minutos)
100000	10^{15}	1000000 ($\approx$ 278 horas)
1000000	10^{18}	91666 horas ($\approx$ 32 años)

Además:

La matriz de $10^6 \times 10^6 = 10^{12}$ elementos, ocupa 8 TB de memoria



Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5. Ch 2.6



Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656. Ch 7.1-5

